



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	KARL GABRIEL R. JARILLA
Section:	AN42
Professor:	MS. CRISOLA TAN
Date:	October 3, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
(Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
 - Classes and objects
 - If-Else statements
 - Loops (for loop)
 - Functions / methods
2. Understand the environment:
 - Two rooms: A and B
 - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output

```
C:\Users\hazel\AppData\Local\Programs\Python\Python38-64\python.exe C:\Users\hazel\AppData\Local\Programs\Python\Python38-64\python.exe
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Press any key to continue . . . |
```

Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
 - o Rule 1: If the current room is dirty, clean it.
 - o Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
 - o The agent's current location
 - o The action taken (cleaned/moved)
 - o Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)

```
C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Press any key to continue . . . |
```

Your Code - Laboratory Tasks

```
class RuleBasedAgent:
    """Class representing the rule-based vacuum cleaner agent."""

    def __init__(self, start_location='A'):
        self.location = start_location

    def perceive_and_act(self, environment):
        """
        Perceives the environment and acts based on predefined rules.
        Rule 1: If current room is dirty, clean it.
        Rule 2: If current room is clean, move to the other room.
        """
        print(f"Agent is in room {self.location}")
```

```

    # Check if the room is dirty
    if environment[self.location].lower() == 'dirty':
        environment[self.location] = 'Clean'
        print(f"Room {self.location} has been cleaned.")
    else:
        self.move()

def move(self):
    """Moves the agent to the other room."""
    next_location = 'B' if self.location == 'A' else 'A'
    print(f"Moving to room {next_location}")
    self.location = next_location

def main():
    # Task 1: Set Up the Environment
    # Initialize the two rooms using a dictionary based on user input
    environment = {
        'A': input("Is room A Dirty or Clean? ").strip().capitalize(),
        'B': input("Is room B Dirty or Clean? ").strip().capitalize()
    }

    # Task 3: Run the Simulation
    try:
        steps = int(input("How many steps should the agent run? "))
    except ValueError:
        print("Invalid input. Defaulting to 5 steps.")
        steps = 5

    # Initialize the agent in Room A
    agent = RuleBasedAgent(start_location='A')

    # Run the agent in a loop for the specified steps
    for step in range(1, steps + 1):
        print(f"Step {step}: ", end="")

        # Task 2: Implement the Rule-Based Agent
        agent.perceive_and_act(environment)

        # Print the current state of the environment
        print(f"Environment state: {environment}")

if __name__ == "__main__":
    main()

```

```

Problems Output Debug Console Terminal Ports
PS D:\Microsoft VS Code> & C:\Users\Gabby\AppData\Local\Microsoft\WindowsApps\python3.11.exe "d:/Intelligent Systems (CS0065)/github files/Technical Assessment 2/Rule-Based Agent Simulation.py"
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean
How many steps should the agent run? 5
Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
PS D:\Microsoft VS Code>

```

(BONUS TASK)

import random

class BonusRuleBasedAgent:

"""Agent that handles three rooms and visualizes movement."""

def __init__(self, start_location='A'):
 self.location = start_location

def display_grid(self, environment):
 """Visualizes the agent and room states using a text-based grid."""
 grid_elements = []
 for room in ['A', 'B', 'C']:
 if self.location == room:
 grid_elements.append(f"[{room}-Agent]")
 else:
 grid_elements.append(f"[{room}-{environment[room]}]")
 print(" ".join(grid_elements))

def perceive_and_act(self, environment):
 """
 Perceives the environment and acts.
 Randomly chooses which dirty room to clean next if current is clean.
 """
 # Print initial state for this step
 self.display_grid(environment)

 # Check if all rooms are clean
 if all(state.lower() == 'clean' for state in environment.values()):
 print("All rooms are clean.\nAgent is idle.")

```

        self.display_grid(environment)
        return

    # Action: Clean or Move
    if environment[self.location].lower() == 'dirty':
        environment[self.location] = 'Clean'
        print(f"Room {self.location} has been cleaned.")
        self.display_grid(environment)
    else:
        # Find all dirty rooms
        dirty_rooms = [room for room, state in environment.items() if state.lower() == 'dirty']

        # Randomly select a dirty room and move
        if dirty_rooms:
            next_room = random.choice(dirty_rooms)
            print(f"Moving to room {next_room}")
            self.location = next_room
            self.display_grid(environment)

def main():
    # Initialize the three rooms
    # Room A is defaulted to 'Clean' based on the TA2 bonus sample output structure
    environment = {
        'A': 'Clean',
        'B': input("Is room B Dirty or Clean? ").strip().capitalize(),
        'C': input("Is room C Dirty or Clean? ").strip().capitalize()
    }

    try:
        steps = int(input("How many steps should the agent run? "))
    except ValueError:
        print("Invalid input. Defaulting to 5 steps.")
        steps = 5

    # Initialize the agent in Room A
    agent = BonusRuleBasedAgent(start_location='A')

    # Run the simulation
    for step in range(1, steps + 1):
        print(f"\nStep {step}:")
        agent.perceive_and_act(environment)

if __name__ == "__main__":
    main()

```

```
Problems  Output  Debug Console  Terminal  Ports

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
PS D:\Microsoft VS Code> & C:\Users\Gabby\AppData\Local\Microsoft\Windows\Terminal\
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty
How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room C
[A-Clean] [B-Dirty] [C-Agent]

Step 2:
[A-Clean] [B-Dirty] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Dirty] [C-Agent]

Step 3:
[A-Clean] [B-Dirty] [C-Agent]
Moving to room B
[A-Clean] [B-Agent] [C-Clean]

Step 4:
[A-Clean] [B-Agent] [C-Clean]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Clean]

Step 5:
[A-Clean] [B-Agent] [C-Clean]
All rooms are clean.
Agent is idle.
[A-Clean] [B-Agent] [C-Clean]
PS D:\Microsoft VS Code> 
```


Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ↓

1. The agent follows two strict rules based on the prompt's instructions: Rule 1 dictates that if the current room is dirty, the agent must clean it; Rule 2 dictates that if the current room is clean, the agent must move to the other room.
2. The agent decides by perceiving the state of the room it currently occupies. If the state evaluates to "dirty," it triggers the cleaning action. If it evaluates to "clean," it defaults to the movement action.
3. Because the agent relies solely on its immediate perception rather than memory, it will perceive the first room as clean and move to the second room. It will then perceive the second room as clean and move back to the first room, resulting in an endless loop of movement without any cleaning for the duration of the simulation.
4. The agent's movement logic would need to be expanded beyond a simple binary toggle (e.g., moving back and forth between A and B). You would need to introduce a pathing mechanism, such as randomly selecting the next dirty room from a list of available rooms or implementing a grid-based coordinate system for the agent to navigate.